



Project Synopsis

AI Powered DevOps Monitoring Platform

Bachelor Of Technology

Computer Science and Engineering

Submitted By – Arpit Sharma

Reg.No – 12306106

Submitted To – Mr. Ajay Kumar Badhan

1. Introduction

DevOps is a modern software development methodology that integrates development and operations to automate software delivery processes. Traditional deployment methods require manual configuration, deployment, monitoring, and infrastructure setup, which increases deployment time and the possibility of errors.

This project aims to implement a complete DevOps workflow by integrating multiple DevOps tools to automate application deployment, monitoring, infrastructure provisioning, and intelligent system analysis. The project demonstrates Continuous Integration and Continuous Deployment (CI/CD), microservices architecture, containerization, orchestration, real-time monitoring, Infrastructure as Code (IaC), and Artificial Intelligence.

A Node.js and Express.js application based on microservices architecture is containerized using Docker, deployed on Kubernetes, automated through Jenkins pipelines, monitored using Prometheus and Grafana, and cloud infrastructure is provisioned using Terraform. Gemini AI is integrated with the monitoring system to analyze real-time CPU, memory, pod, and container metrics and provide intelligent insights through an AI-powered DevOps assistant.

2. Problem Statement

In traditional software deployment environments:

- Application deployment is performed manually.
- Monitoring systems are limited.
- Infrastructure setup takes more time.
- Scaling applications is difficult.
- Deployment failures may cause downtime.
- Manual processes increase human errors.
- Managing multiple application services independently can be complex.
- Monitoring data requires manual analysis to identify system issues.

3. Objective of the Project

The objectives of the project are:

- To automate application deployment using CI/CD pipelines.
- To containerize applications using Docker.
- To orchestrate containers using Kubernetes.
- To automate deployment workflows using Jenkins.
- To monitor Kubernetes clusters and applications in real time.
- To visualize system metrics using Grafana dashboards.
- To provision cloud infrastructure using Terraform.
- To implement scalable and self-healing deployment architecture.
- To integrate Gemini AI for intelligent analysis of application and infrastructure metrics.
- To provide an AI-powered assistant for DevOps and infrastructure-related queries.

4. Proposed System

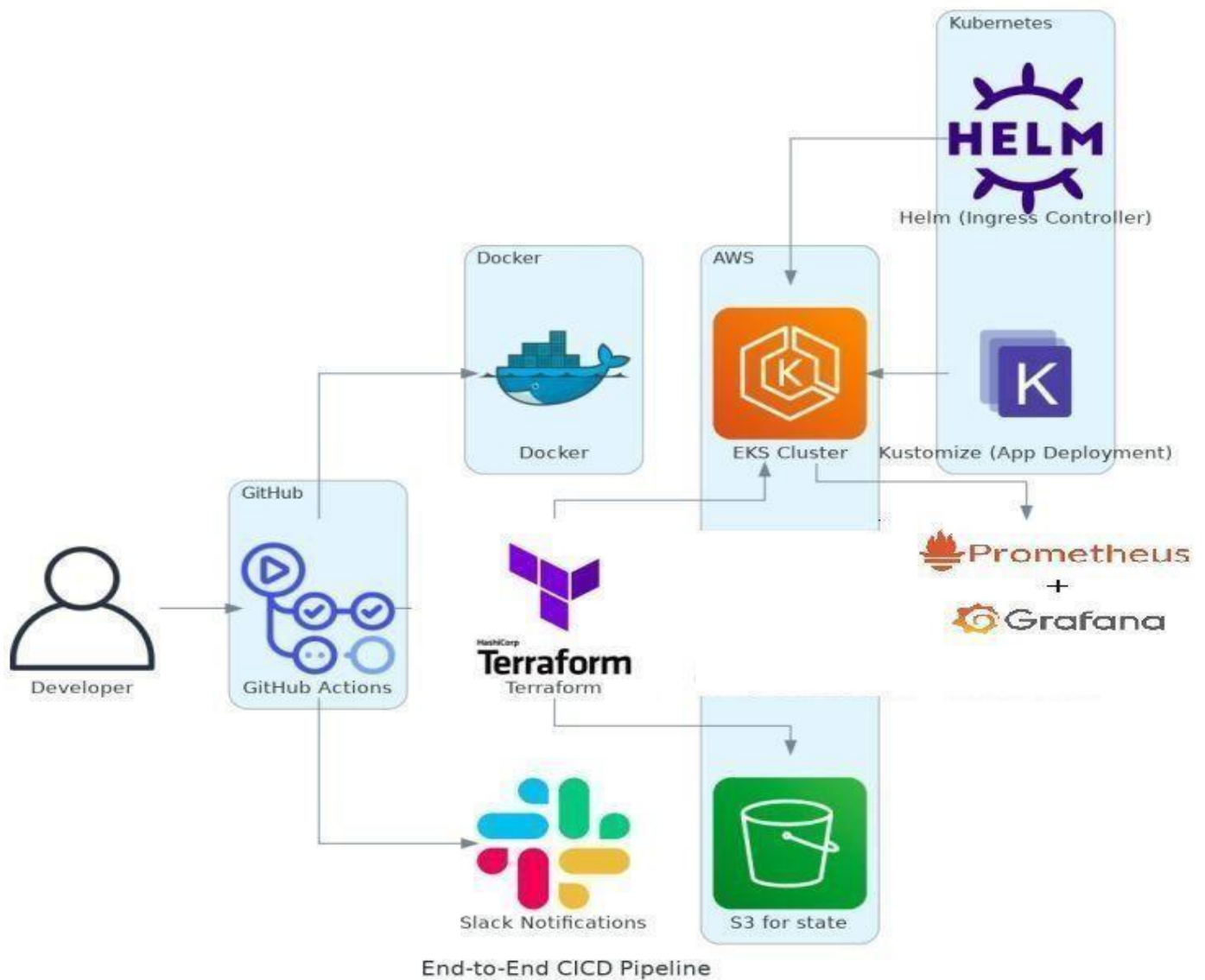
The proposed system consists of a complete DevOps pipeline that automates software delivery from source code management to deployment and monitoring.

Initially, a Node.js application is developed and based on microservices architecture and stored in a GitHub repository. Jenkins automatically detects code changes and triggers the CI/CD pipeline. The pipeline builds a Docker image and pushes it to Docker Hub. Kubernetes then deploys the containerized application and manages scaling and self-healing.

Prometheus continuously collects performance metrics such as CPU usage, memory usage, pod status, and node health from the Kubernetes cluster. Grafana visualizes these metrics using real-time dashboards. Terraform is used to provision AWS cloud infrastructure using Infrastructure as Code.

Gemini AI is integrated with the monitoring system to analyze the collected infrastructure and application metrics. The AI-powered assistant can provide intelligent insights and answer DevOps and infrastructure-related queries based on real-time CPU, memory, pod, and container information.

5. System Architecture



6. Workflow of the Project

The workflow of the project is as follows:

Step 1 — Application Development

A simple Node.js web application is created using Express.js and structured as multiple independent microservices.

Step 2 — Containerization

The application is containerized using Docker by creating a Dockerfile and Docker image.

Step 3 — Docker Hub Integration

The Docker image is pushed to Docker Hub for centralized image storage.

Step 4 — Kubernetes Deployment

The microservices are deployed on Kubernetes using Deployment and Service YAML files.

Step 5 — CI/CD Pipeline

Jenkins automates the build, push, and deployment process through a Jenkins pipeline.

Step 6 — Monitoring Setup

Prometheus collects metrics from Kubernetes and Grafana displays them on dashboards.

Step 7 — Infrastructure Automation

Terraform uses the Kreuzwerker Docker provider to automate Docker image and container provisioning through Infrastructure as Code, enabling consistent and repeatable local infrastructure setup.

Step 8 — AI-Based Monitoring Analysis

Gemini AI analyzes real-time monitoring information such as CPU usage, memory utilization, pod status, and container metrics. The AI-powered assistant provides intelligent insights and answers DevOps and infrastructure-related queries.

7. Tools and Technologies Used

Purpose

• Docker	Application Containerization
• Kubernetes	Container orchestration
• Jenkins	CI/CD automation
• Prometheus	Metrics Collection
• Grafana	Dashboard Visualization
• Terraform	Infrastructure Provisioning
• GitHub	Source code management
• Node.js	Backend Application
• Express.js	Web Framework
• Docker Hub	Container image repository
• Microservices	Independent application architecture
• Gemini AI	Intelligent monitoring and AI analysis

8. Advantages

- Reduces manual deployment effort
- Improves software delivery speed
- Enables scalable deployments
- Supports automated monitoring
- Reduces downtime through self-healing
- Simplifies infrastructure management
- Improves deployment reliability
- Demonstrates real-world DevOps practices

9. Future Enhancements

The following features can be added in the future:

- Auto-scaling based on traffic load
- Alert notification system
- Load balancing
- Multi-node Kubernetes cluster

- Centralized logging using ELK stack
- Deployment on managed Kubernetes services such as Amazon EKS
- Advanced security integration

10. Conclusion

The AI Powered DevOps Monitoring Platform using microservices successfully demonstrates the implementation of modern DevOps practices using industry-standard tools and technologies.

The project automates application deployment, monitoring, and cloud infrastructure provisioning using Docker, Kubernetes, Jenkins, Prometheus, Grafana, and Terraform.

The system provides automated deployment, scalability, monitoring, AI-based analysis, and infrastructure management capabilities, making it suitable for modern cloud-native software environments and real-world DevOps implementations.